

TECHNICAL SECURITY REMEDIATION REPORT

wazosports.com | API Security Vulnerabilities & Fixes

Platform Express.js / React (Vite) / Replit	Assessment Date 1 May 2026	Document Type Technical Remediation Guide
---	--------------------------------------	---

1. Architecture Overview

The wazosports.com platform is a full-stack JavaScript application hosted on Replit and served via Google Cloud infrastructure. Understanding the architecture is essential to correctly implementing the remediations described in this document.

1.1 Exposed API Surface

The following endpoints were confirmed as publicly accessible without authentication:

```
GET /api/questions -> Returns ALL questions with correct answers
GET /api/questions/all -> Full question bank with answers
GET /api/prizes -> Prize pool with probability weights
GET /api/prizes/all -> Same
GET /api/admin/users -> FULL user PII database (100+ records)
GET /api/admin/payments -> All payment transactions + virtual account numbers
GET /api/admin/wallets -> All user wallet balances
GET /api/admin/winners -> Winner records with PII
GET /api/admin/quiz-sessions -> All quiz session history
GET /api/admin/referrals -> Full referral network graph
GET /api/admin/stats -> Platform aggregate stats
POST /api/quiz/start -> Accepts arbitrary userId (no auth)
POST /api/wheel/spin -> No auth check on userId
POST /api/payments/create-virtual-account -> No auth
```

2. Critical Fix: Authentication Middleware

The root cause of nearly all critical findings is the absence of authentication middleware on admin routes. The fix must be applied at the Express router level before route handlers are registered.

2.1 Step 1 — Create Authentication Middleware

Create a new file: server/middleware/auth.js

```
// server/middleware/auth.js
const jwt = require('jsonwebtoken');

// Middleware: require valid JWT for any authenticated route
const requireAuth = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Authentication required' });
  }
  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
};

// Middleware: require admin role
const requireAdmin = (req, res, next) => {
  if (!req.user || !req.user.isAdmin) {
    return res.status(403).json({ error: 'Admin access required' });
  }
  next();
};

module.exports = { requireAuth, requireAdmin };
```

2.2 Step 2 — Apply Middleware to Admin Routes

In your main Express app file (server/index.js or app.js), apply the admin middleware to ALL /api/admin/* routes:

```
// server/index.js (or app.js)
const { requireAuth, requireAdmin } = require('./middleware/auth');

// BEFORE registering any admin routes:
// This single line protects ALL /api/admin/* endpoints
app.use('/api/admin', requireAuth, requireAdmin);

// Now register your admin routes AFTER the middleware
app.use('/api/admin', adminRouter);

// For user routes that need auth but not admin:
app.use('/api/quiz', requireAuth);
app.use('/api/wheel', requireAuth);
app.use('/api/payments', requireAuth);
```

IMPORTANT: The `app.use('/api/admin', requireAuth, requireAdmin)` line **MUST** appear **BEFORE** your route definitions. Express processes middleware in order of registration.

2.3 Step 3 — Secure Environment Variables

Add a strong `JWT_SECRET` to your Replit Secrets (not `.env` file):

```
# In Replit: Go to Tools > Secrets > Add new secret
# Key: JWT_SECRET
# Value: (generate with: node -e
"console.log(require('crypto').randomBytes(64).toString('hex'))")

# Also add:
# Key: PAYSTACK_SECRET_KEY
# Value: your_paystack_secret_key

# In code, access as:
process.env.JWT_SECRET
process.env.PAYSTACK_SECRET_KEY
```

3. Fix: Remove Answer Key from Public API

The `/api/questions` endpoint returns the `correctAnswer` field to all users. This must be stripped from the public response. The correct answer should only be sent to validate a specific submitted answer, not pre-loaded.

3.1 Option A — Strip Field in Route Handler (Quick Fix)

```
// In your questions route handler:
app.get('/api/questions', async (req, res) => {
  const questions = await db.query('SELECT * FROM questions WHERE is_active = true');

  // Strip correctAnswer before sending to client
  const safeQuestions = questions.rows.map(q => {
    const { correctAnswer, ...safeQuestion } = q; // Destructure to exclude
    return safeQuestion;
  });

  res.json(safeQuestions);
});
```

3.2 Option B — Separate Answer Validation Endpoint (Best Practice)

```
// Create a dedicated answer check endpoint (requires auth)
app.post('/api/quiz/answer', requireAuth, async (req, res) => {
  const { questionId, selectedAnswer, sessionId } = req.body;
  const userId = req.user.id; // From JWT, not request body

  // Fetch correct answer server-side only
  const question = await db.query(
    'SELECT correct_answer FROM questions WHERE id = $1',
    [questionId]
  );

  const isCorrect = question.rows[0].correct_answer === selectedAnswer;

  // Update session, award points, etc.
  await db.query(
    'UPDATE quiz_sessions SET correct_answers = correct_answers + $1 WHERE id = $2 AND user_id = $3',
    [isCorrect ? 1 : 0, sessionId, userId]
  );

  res.json({ correct: isCorrect }); // Never return the answer
});
```

4. Fix: Secure Quiz Start & Wheel Spin

The `/api/quiz/start` endpoint currently accepts a `userId` in the request body with no verification. This allows anyone to start a quiz session on behalf of another user. The `userId` must always come from the verified JWT token.

4.1 Quiz Start — Bind to Authenticated User

```
// BEFORE (vulnerable):
app.post('/api/quiz/start', async (req, res) => {
  const { userId } = req.body; // DANGER: User-controlled
  // ...
});

// AFTER (secure):
app.post('/api/quiz/start', requireAuth, async (req, res) => {
  const userId = req.user.id; // From JWT token, cannot be spoofed

  // Check if user has paid (verify payment before allowing quiz)
  const wallet = await db.query(
    'SELECT balance FROM wallets WHERE user_id = $1',
    [userId]
  );
  if (!wallet.rows[0] || wallet.rows[0].balance < 100) {
    return res.status(402).json({ error: 'Insufficient balance' });
  }

  // Create session bound to this user
  const session = await db.query(
    'INSERT INTO quiz_sessions (user_id, status) VALUES ($1, $2) RETURNING id',
    [userId, 'in_progress']
  );
  res.json({ sessionId: session.rows[0].id });
});
```

4.2 Wheel Spin — Prevent Free Spins

```
app.post('/api/wheel/spin', requireAuth, async (req, res) => {
  const userId = req.user.id; // From JWT

  // Atomically check and deduct points (prevents race conditions)
  const result = await db.query(`
    UPDATE wallets
    SET points = points - 5
    WHERE user_id = $1 AND points >= 5
    RETURNING points
  `, [userId]);

  if (result.rowCount === 0) {
    return res.status(402).json({ error: 'Insufficient points' });
  }

  // Proceed with spin logic...
});
```

5. Fix: Security Headers

The platform is missing several important HTTP security headers. Add the helmet package to automatically configure these:

5.1 Install and Configure Helmet

```
# Install helmet
npm install helmet
```

```
// server/index.js
const helmet = require('helmet');

// Add BEFORE all routes
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self'],
      styleSrc: ['self', 'unsafe-inline'],
      imgSrc: ['self', 'data:', 'https:'],
      connectSrc: ['self'],
      fontSrc: ['self'],
      objectSrc: ['none'],
      upgradeInsecureRequests: [],
    },
  },
}));

// Remove X-Powered-By header (also done by helmet, but explicit is better)
app.disable('x-powered-by');
```

5.2 Add Rate Limiting

```
npm install express-rate-limit
```

```
const rateLimit = require('express-rate-limit');

// General API limiter
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // max 100 requests per window
  message: { error: 'Too many requests, please try again later.' }
});

// Strict limiter for auth endpoints
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10, // max 10 login attempts per 15 min
  message: { error: 'Too many login attempts.' }
});

app.use('/api/', apiLimiter);
```

```
app.use('/api/auth/login', authLimiter);  
app.use('/api/auth/signup', authLimiter);
```

6. Fix: Input Validation

The quiz start endpoint uses Zod for validation but still accepts user-supplied `userId`. All endpoints that create, read, update, or delete data must validate all inputs server-side.

6.1 Validate Payment Creation

```
const { z } = require('zod');

const createVirtualAccountSchema = z.object({
  amount: z.number().int().min(100).max(100000),
  // userId and userName come from JWT, not request body
});

app.post('/api/payments/create-virtual-account', requireAuth, async (req, res) => {
  const parsed = createVirtualAccountSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(400).json({ error: parsed.error.issues });
  }

  const { amount } = parsed.data;
  const userId = req.user.id; // From JWT
  const userName = req.user.fullName; // From JWT

  // Proceed with payment creation...
});
```

7. Replit-Specific Security Configuration

Because the platform is hosted on Replit, there are specific configuration steps to take within the Replit environment.

7.1 Use Replit Secrets (Not .env Files)

Never store secrets in .env files on Replit as they can be exposed. Use Replit's built-in Secrets manager:

- Go to your Repl > Tools > Secrets
- Add each secret as a key-value pair
- Access in code via `process.env.SECRET_NAME`
- Ensure your .gitignore excludes .env if you have one

Required secrets to configure:

```
JWT_SECRET=<64-byte random hex string>
DATABASE_URL=<your database connection string>
PAYSTACK_SECRET_KEY=<your paystack secret key>
PAYSTACK_PUBLIC_KEY=<your paystack public key>
NODE_ENV=production
```

7.2 Set NODE_ENV to Production

Ensure NODE_ENV is set to 'production' in Replit Secrets. This disables development error messages that reveal stack traces and internal paths.

```
// In Express, check NODE_ENV for error handling:
app.use((err, req, res, next) => {
  const isDev = process.env.NODE_ENV === 'development';
  res.status(err.status || 500).json({
    error: isDev ? err.message : 'Internal server error',
    // Never expose stack traces in production
  });
});
```

7.3 Restrict Replit Project Visibility

- Set your Repl to Private (not Public) in Replit settings
- Ensure no sensitive files are visible in the file explorer
- Do not publish the Repl URL publicly
- Use Replit's Custom Domain feature to serve via wazosports.com only

8. Additional Security Best Practices

8.1 Database Query Security — Prevent SQL Injection

Always use parameterised queries. Never concatenate user input into SQL strings:

```
// VULNERABLE - Never do this:
const q = `SELECT * FROM users WHERE email = '${email}'`;

// SAFE - Always use parameterised queries:
const q = await db.query(
  'SELECT * FROM users WHERE email = $1',
  [email] // Parameter, not string interpolation
);
```

8.2 Password Security

```
npm install bcrypt
```

```
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 12;

// On registration:
const hashedPassword = await bcrypt.hash(plainTextPassword, SALT_ROUNDS);
// Store hashedPassword in database, never plaintext

// On login:
const isValid = await bcrypt.compare(submittedPassword, storedHash);
```

8.3 CORS Configuration

```
npm install cors
```

```
const cors = require('cors');

app.use(cors({
  origin: ['https://wazosports.com'], // Only allow your domain
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));
// Remove the wildcard: app.use(cors()) is INSECURE
```

8.4 Implement Logging and Monitoring

```
npm install winston morgan
```

```
const morgan = require('morgan');
const winston = require('winston');

// Request logging
app.use(morgan('combined'));

// Log suspicious patterns:
// - Multiple failed login attempts from same IP
// - Requests to /api/admin/* from non-admin users
// - Unusual payment request volumes
// - Quiz sessions with 100% correct rate (answer key abuse)
```

8.5 Implement Paystack Webhook Verification

Currently payment confirmation may not be cryptographically verified. Validate all Paystack webhooks:

```
const crypto = require('crypto');

app.post('/api/paystack/webhook', express.raw({type: 'application/json'}), (req, res) => {
  const hash = crypto
    .createHmac('sha512', process.env.PAYSTACK_SECRET_KEY)
    .update(req.body) // Raw body, not parsed JSON
    .digest('hex');

  if (hash !== req.headers['x-paystack-signature']) {
    return res.status(401).json({ error: 'Invalid signature' });
  }

  // Now safe to process payment event
  const event = JSON.parse(req.body);
  // ...
  res.sendStatus(200);
});
```

9. Remediation Checklist

Use this checklist to track implementation progress. All CRITICAL items must be completed before the platform accepts new user registrations or payments.

	Task	Priority	Done
1	Add requireAuth + requireAdmin middleware to all /api/admin/* routes	CRITICAL	☐
2	Add requireAuth middleware to /api/quiz/*, /api/wheel/*, /api/payments/*	CRITICAL	☐
3	Remove correctAnswer field from public /api/questions response	CRITICAL	☐
4	Replace userId from req.body with req.user.id from JWT in all routes	CRITICAL	☐
5	Add JWT_SECRET, DB credentials to Replit Secrets (not code)	CRITICAL	☐
6	Install and configure helmet for security headers	HIGH	☐
7	Install and configure express-rate-limit on all endpoints	HIGH	☐
8	Restrict CORS to wazosports.com origin only	HIGH	☐
9	Set NODE_ENV=production in Replit Secrets	HIGH	☐
10	Implement Paystack webhook signature verification	HIGH	☐
11	Set Replit project to Private	HIGH	☐
12	Verify all SQL queries use parameterised inputs	HIGH	☐
13	Ensure passwords are hashed with bcrypt (saltRounds >= 12)	HIGH	☐
14	Add request logging with morgan/winston	MEDIUM	☐
15	Implement quiz session rate limiting (max N sessions per day per user)	MEDIUM	☐
16	Add anomaly detection for 100% correct quiz sessions	MEDIUM	☐
17	Set up automated vulnerability scanning in Replit CI/CD	MEDIUM	☐
18	Conduct post-fix security validation (re-test all endpoints)	HIGH	☐

10. Post-Fix Validation

After implementing all fixes, re-test each endpoint to confirm the vulnerabilities are resolved. All of the following commands should return 401 or 403, not data:

```
# All of these should return 401/403 after fixes are applied:

curl https://wazosports.com/api/admin/users
# Expected: {"error": "Authentication required"}

curl https://wazosports.com/api/admin/payments
# Expected: {"error": "Authentication required"}

curl https://wazosports.com/api/admin/wallets
# Expected: {"error": "Authentication required"}

curl https://wazosports.com/api/admin/winners
# Expected: {"error": "Authentication required"}

curl https://wazosports.com/api/questions | python3 -c "import sys,json;
q=json.load(sys.stdin); print('FAIL: correctAnswer exposed' if 'correctAnswer' in
q[0] else 'PASS: correctAnswer removed')"
```

```
# Confirm X-Powered-By is gone:
curl -I https://wazosports.com | grep -i 'x-powered-by'
# Expected: no output

# Confirm CSP header is present:
curl -I https://wazosports.com | grep -i 'content-security-policy'
# Expected: content-security-policy: default-src 'self'...
```

After all fixes are verified, conduct a full responsible disclosure notification to your users explaining that a security issue was identified and resolved, and advising them to change their passwords and monitor their bank accounts as a precaution.

10.1 Ongoing Security Practices

- Run npm audit weekly to check for vulnerable dependencies
- Subscribe to Node.js and Express.js security advisories
- Review all new API endpoints for authentication before deployment
- Implement a staging environment to test security changes before production